Programação para Dispositivos Móveis

React Native — Views e Flexbox

Lista de Exercícios Práticos

Disciplina: Desenvolvimento de Sistemas — Desenvolvimento Mobile

Conteúdo: View, Flexbox, Layouts

Total: 10 exercícios

Legenda de dificuldade:

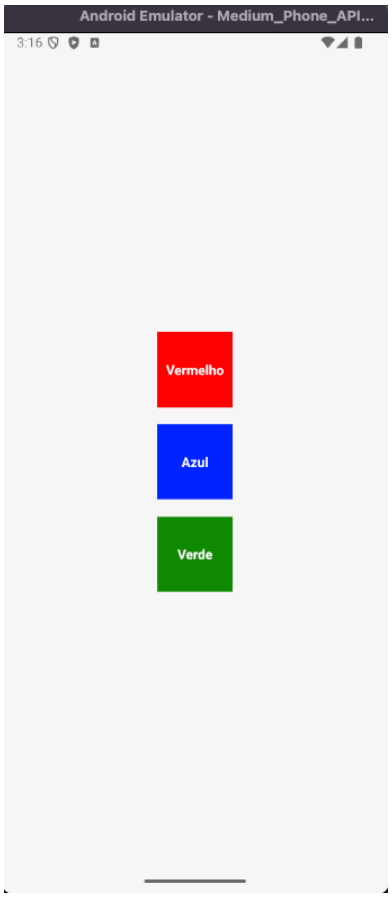
🟢 Fácil · 🟡 Médio · 🔴 Difícil

Exercício 1 — Coluna de Três Caixas 🟢

Objetivo: Praticar layout vertical básico com Views.

Crie um layout com três caixas coloridas (vermelha, azul e verde) empilhadas verticalmente, uma abaixo da outra. Cada caixa deve ter altura fixa de 80 e 80 de largura.

Layout Esperado:



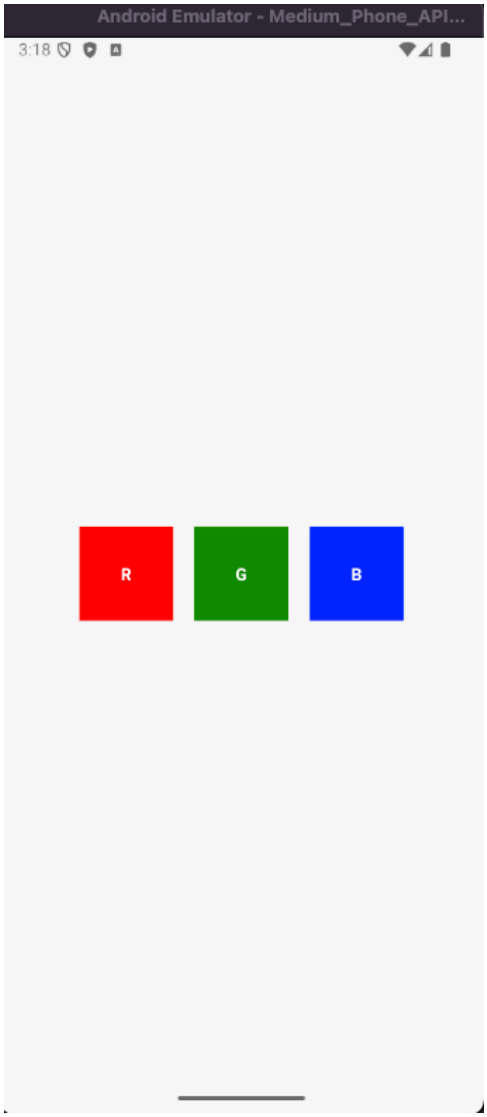
> **Dica:** Use `flexDirection: 'column'` (padrão) no container e defina `height` em cada caixa.

Exercício 2 — Três Caixas em Linha 🟢

Objetivo: Praticar layout horizontal com `flexDirection: 'row'`.

Crie um layout com três caixas quadradas (80×80) dispostas lado a lado horizontalmente. Use as cores vermelho, azul e verde.

Layout Esperado:



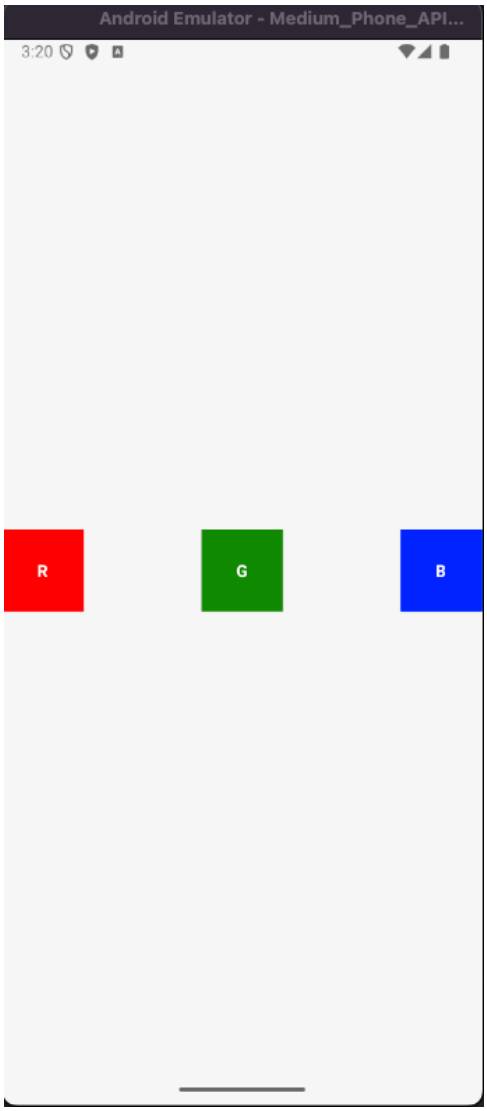
> **Dica:** Use `flexDirection: 'row'` no container pai.

Exercício 3 — Espaçamento com space-between 🟢

Objetivo: Entender `justifyContent` e distribuição de espaço.

Crie três caixas quadradas (70×70) em uma linha horizontal. O container deve distribuir o espaço igualmente entre as caixas usando `justifyContent`. A primeira caixa deve ficar encostada à esquerda e a última à direita.

Layout Esperado:



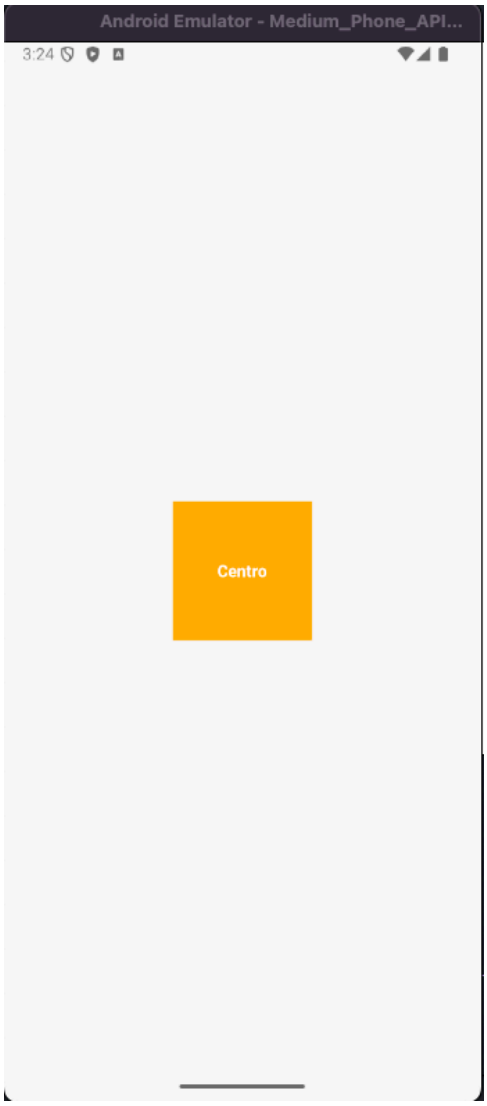
> **Dica:** Combine `flexDirection: 'row'` com `justifyContent: 'space-between'`.

Exercício 4 — Caixa Centralizada 🟢

Objetivo: Dominar centralização com `justifyContent` e `alignItems`.

Crie um layout onde uma única caixa (120×120) fique perfeitamente centralizada na tela, tanto na horizontal quanto na vertical. O container deve ocupar a tela toda.

Layout Esperado:



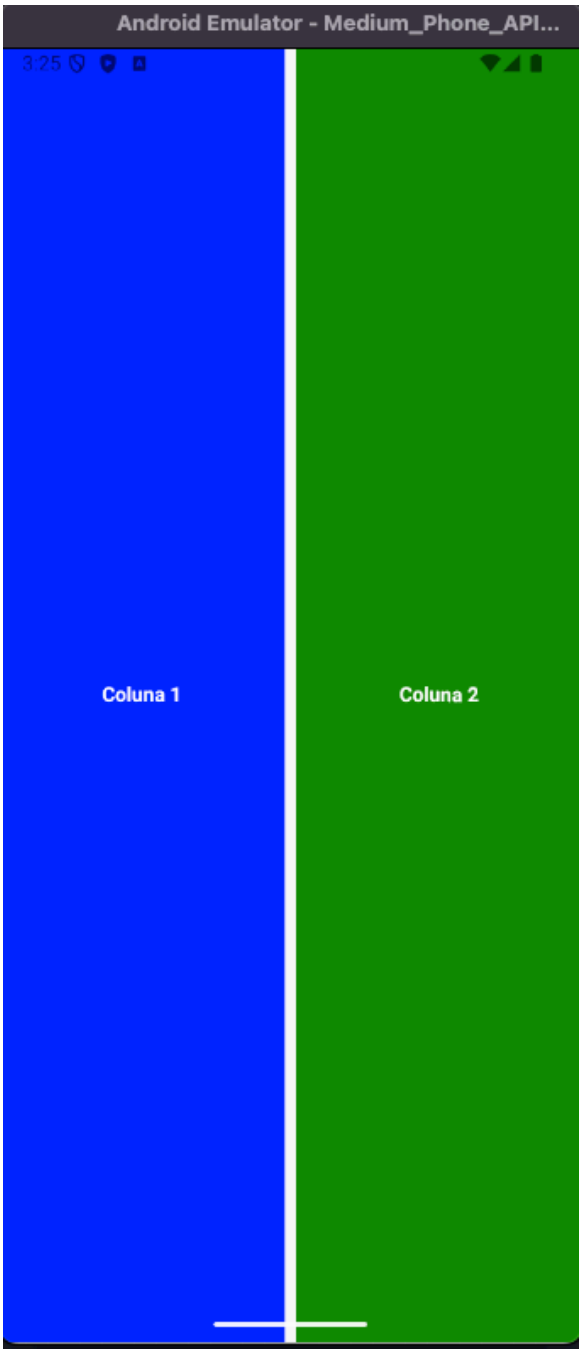
> **Dica:** O container precisa de `flex: 1`, `justifyContent: 'center'` e `alignItems: 'center'`.

Exercício 5 — Layout de Duas Colunas 🟡

Objetivo: Usar `flex: 1` para dividir espaço proporcionalmente.

Crie um layout dividido em duas colunas de largura igual, lado a lado, cada uma com uma cor diferente. Ambas devem ocupar toda a altura disponível. Use um gap de 8 entre elas.

Layout Esperado:



> **Dica:** Use `flexDirection: 'row'` no pai, e `flex: 1` em cada coluna. Para o gap, use a propriedade `gap` ou `margin`.

Exercício 6 — Header, Conteúdo e Footer 🟡

Objetivo: Criar layouts de página com áreas fixas e flexíveis.

Crie uma estrutura de página com três seções: um header fixo no topo (altura 60), uma área de conteúdo que ocupa todo o espaço restante, e um footer fixo na parte inferior (altura 50). O container principal deve ocupar toda a tela.

Layout Esperado:

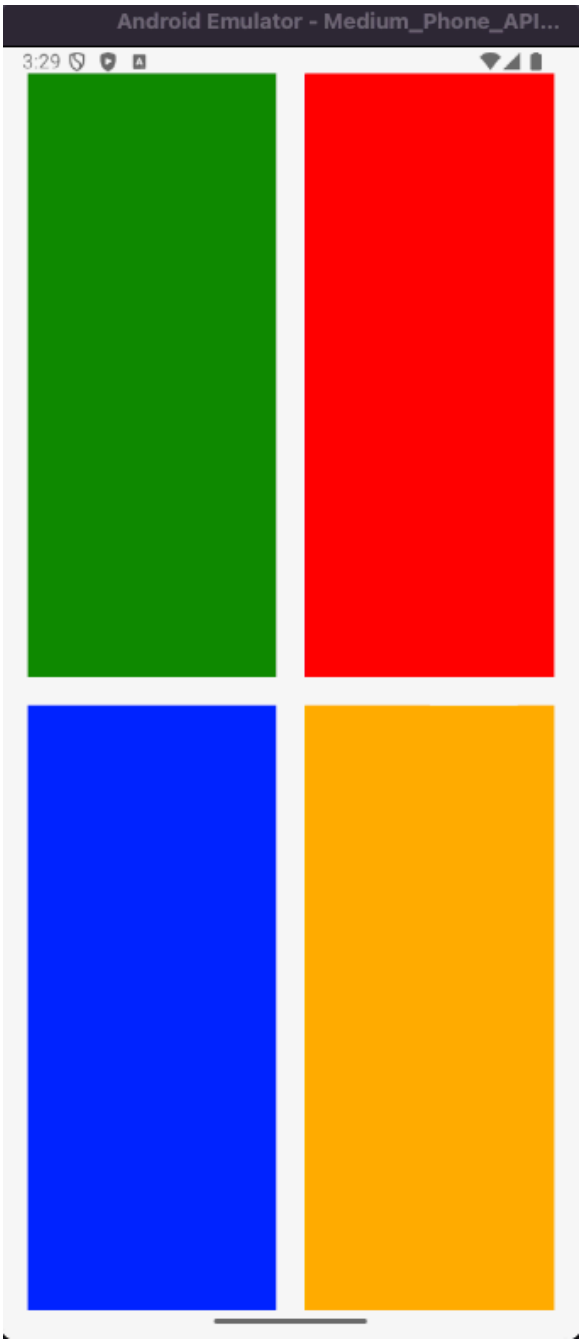
> **Dica:** O container usa `flex: 1`. Header e Footer têm altura fixa. O conteúdo usa `flex: 1` para preencher o restante.

Exercício 7 — Grid 2x2 🟡

Objetivo: Construir layouts tipo grid combinando `row` e `column`.

Crie um grid de 2 linhas e 2 colunas. Cada célula deve ter uma cor diferente e todas devem ter o mesmo tamanho, preenchendo a tela uniformemente. Use um gap de 10 entre as células e um padding de 20 no contêiner principal.

Layout Esperado:



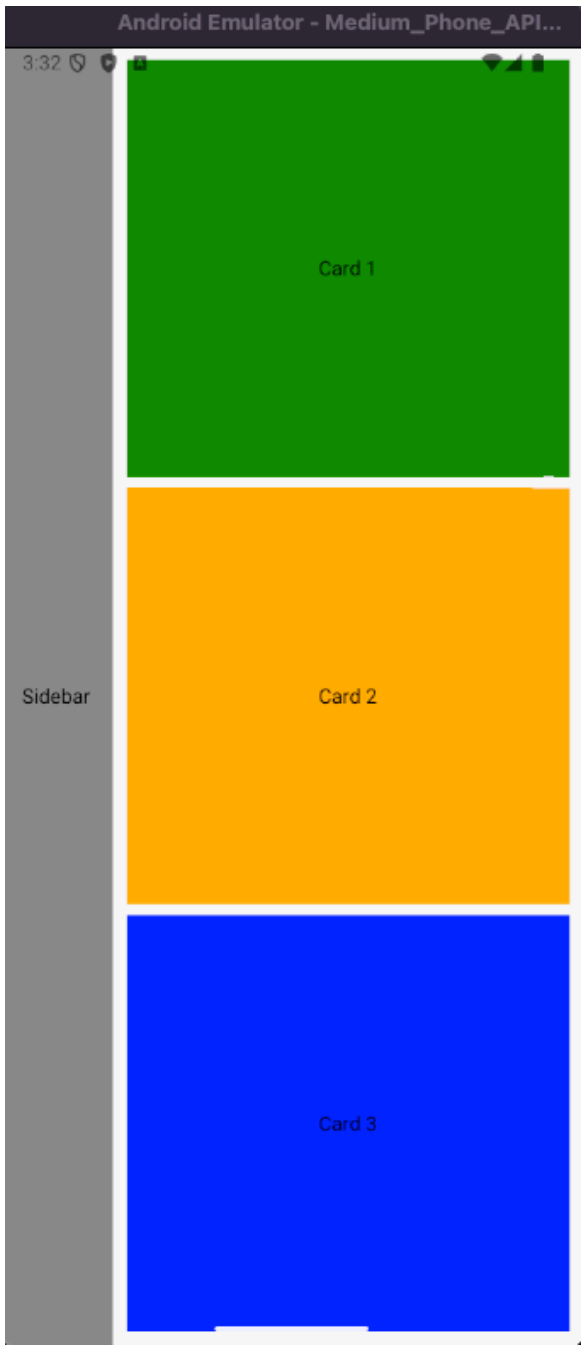
> **Dica:** Crie duas Views com `flexDirection: 'row'`, cada uma representando uma linha. Dentro de cada linha, coloque duas Views com `flex: 1`.

Exercício 8 — Sidebar com Área de Conteúdo 🟡

Objetivo: Combinar largura fixa com largura flexível em layouts horizontais.

Crie um layout com uma barra lateral (sidebar) fixa à esquerda com largura 80 e cor escura. À direita, crie uma área de conteúdo que ocupa o restante da largura, contendo três cards empilhados verticalmente com cores diferentes e altura fixa de 100 cada. Use padding de 10 na área de conteúdo.

Layout Espresso:



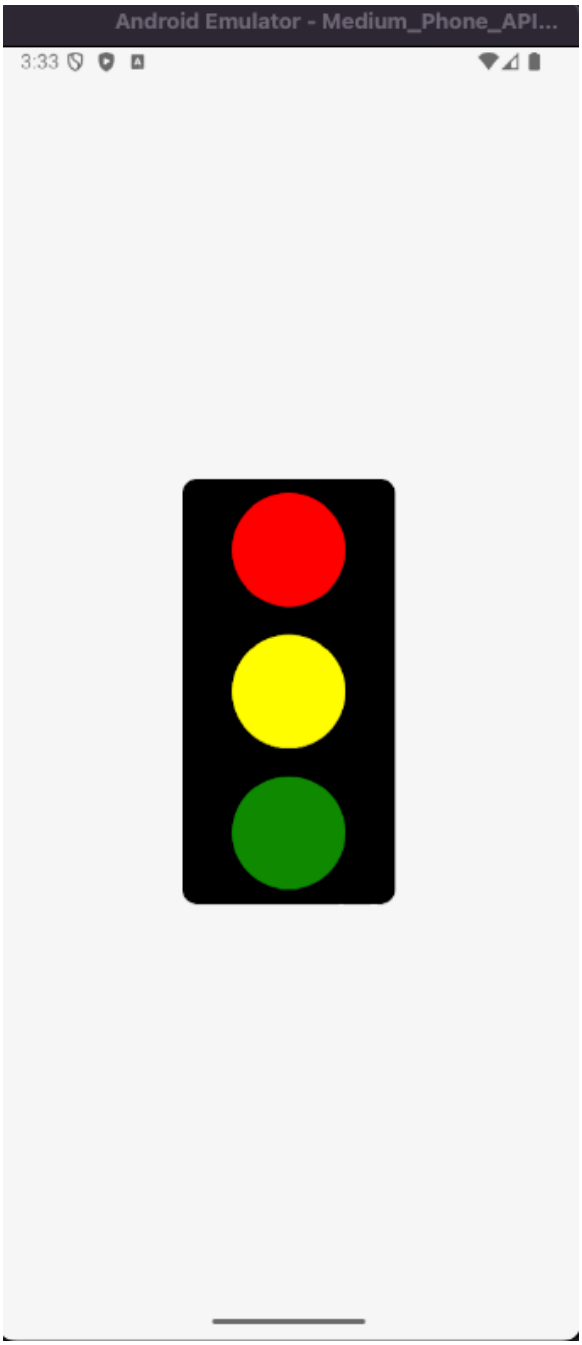
> **Dica:** Use `flexDirection: 'row'` no pai. A sidebar tem `width` fixo. A área de conteúdo usa `flex: 1` com `padding`.

Exercício 9 — Semáforo 🔴

Objetivo: Combinar centralização, padding, `borderRadius` e espaçamento.

Crie um semáforo vertical: um retângulo escuro (arredondado) centralizado na tela contendo três círculos (vermelho, amarelo, verde) empilhados com espaçamento uniforme entre eles. Os círculos devem ter 80×80 com `borderRadius: 40`.

Layout Esperado:



> **Dica:** O container externo centraliza tudo. O retângulo escuro usa `padding`, `alignItems: 'center'` e `justifyContent: 'space-evenly'`. Use `borderRadius` para arredondar.

Exercício 10 — Mini Dashboard 🔴

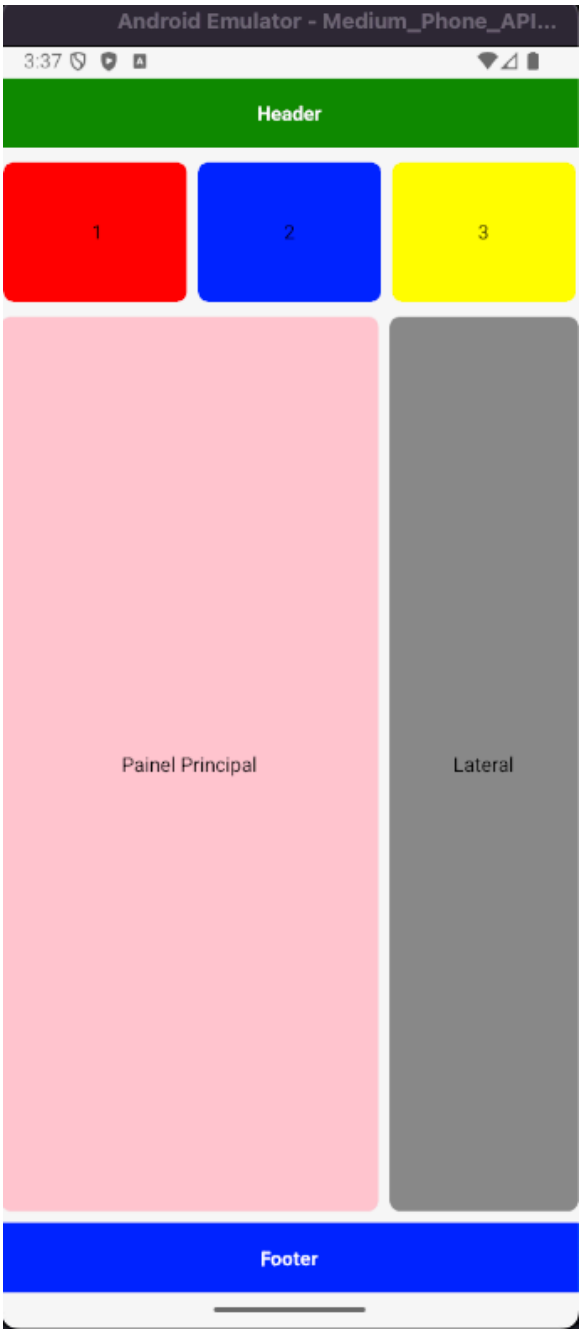
Objetivo: Integrar todas as técnicas de Flexbox em um layout complexo e realista.

Construa um layout de dashboard completo com:

- Um **header** (altura 50, cor azul escuro);
- Uma **linha com 3 cards** de tamanho igual com gap de 8;
- Abaixo, uma **linha com um painel principal** (`flex: 2`) e um **painel lateral** (`flex: 1`) com gap de 8;
- Um **footer** (altura 40, cor escura).

Todas as seções devem ter padding de 8. O conteúdo central deve expandir para preencher o espaço restante.

Layout Esperado:



> **Dica:** Estruture como: `View(flex:1)` > Header + `View(flex:1, conteúdo)` + Footer. Dentro do conteúdo, crie as linhas com `flexDirection: 'row'`. Nos cards da primeira linha use `flex: 1`. Na segunda linha, use `flex: 2` e `flex: 1`.

Execução e testes

- Iniciar servidor + emulador a partir do domando `npx expo start`

- Criar os componentes de exercício
- Chamar o componente no arquivo `App.js`

Referência Rápida — Propriedades Flexbox

Propriedade	Valores comuns	Descrição
`flexDirection`	`'column'` | `'row'`	Define o eixo principal (vertical ou horizontal)
`justifyContent`	`'flex-start'` | `'center'` | `'flex-end'` | `'space-between'` | `'space-around'` | `'space-evenly'`	Alinha filhos no eixo principal
`alignItems`	`'flex-start'` | `'center'` | `'flex-end'` | `'stretch'`	Alinha filhos no eixo transversal
`flex`	`1, 2, 3...`	Proporção do espaço disponível que o item ocupa
`gap`	`8, 10, 16...`	Espaçamento entre os filhos
`padding`	`8, 10, 16...`	Espaçamento interno do container
`margin`	`8, 10, 16...`	Espaçamento externo do elemento
`width` / `height`	`80, 120...`	Dimensões fixas do elemento
`borderRadius`	`4, 8, 40...`	Arredondamento das bordas

Exemplo de Estrutura Básica

```
import { View, StyleSheet } from 'react-native';

export default function App() {
  return (
    <View style={styles.container}>
```

```
      <View style={styles.box1} />
      <View style={styles.box2} />
      <View style={styles.box3} />
    </View>
  );
}

const styles = StyleSheet.create({
  container: {
    flex: 1,
    flexDirection: 'column',
    justifyContent: 'center',
    alignItems: 'center',
    backgroundColor: '#F5F5F5',
  },
  box1: { width: 100, height: 100, backgroundColor: 'red' },
  box2: { width: 100, height: 100, backgroundColor: 'blue' },
  box3: { width: 100, height: 100, backgroundColor: 'green'
},
});
```

Bons estudos!